



cpi'fp

Los Enlaces

DESARROLLO WEB EN ENTORNO SERVIDOR

2º DESARROLLO DE APLICACIONES WEB

BLADE (VISTAS Y PLANTILLAS)

1. BLADE

Blade es un poderoso y sencillo sub-lenguaje que nos permitirá generar **plantillas de vistas** para minimizar el código que necesitaremos para nuestras vistas.

Las plantillas Blade admiten condiciones y bucles para operar con las variables PHP, de modo que la misma plantilla se comporta de forma diferente con diferentes conjuntos de datos. ¡Y se acabó la pesadilla de abrir y cerrar comillas en las instrucciones `echo`!

1. BLADE

Con Blade, tampoco será necesario abrir y cerrar php (`<?php ... ?>`) para operar con las variables del servidor y generar la salida.

El código escrito con Blade no solo es más limpio y fácil de depurar que con PHP básico, sino también más seguro, porque Blade impide cualquier [ataque con XSS](#).

Pero eso no es todo. Además, unas plantillas pueden heredar de otras para reutilizar mejor el código.

2. MASTER LAYOUT

El ***master layout*** es el diseño maestro del que derivan todas las vistas. Las aplicaciones web suelen tener un *master layout* muy definido, y todas las vistas de la aplicación lo respetan.

Ese *master layout* suele almacenarse, por convenio, en un archivo llamado `/resources/views/layouts/master.blade.php`

Vamos a construir uno de ejemplo. Será muy simple, pero más adelante puedes complicarlo todo lo que quieras, y cada modificación afectará automáticamente a todas las vistas de tu aplicación.

2. MASTER LAYOUT

```
<html>
  <head> <title>@yield('title')</title> </head>
  <body>
    @section('sidebar')
      Este es mi master sidebar.
    @show
    <div class="container">
      @yield('content')
    </div>
  </body>
</html>
```

2. MASTER LAYOUT

Aquí nos encontramos con varias **directivas** importantes de Blade:

@yield('Titulo') → Esto define una sección llamada 'Titulo', que en el master layout está vacía. Las vistas que hereden del master layout tendrán que definir cuál es el valor de 'Titulo'.

@section('sidebar') → Esto define una sección llamada 'sidebar' que NO está vacía (contiene el texto "Este es mi master sidebar", pero podría contener cualquier otra cosa). Las vistas que hereden del master layout podrán **AÑADIR** contenido a esta sección.

2. MASTER LAYOUT

Directivas:

@show → Hace que el contenido de la sección se muestre en ese punto del documento.

@yield('content') → De nuevo, una sección que está vacía en el master layout y cuyo contenido tendrá que definirse en las vistas que hereden de ésta.

2. MASTER LAYOUT

Después de crear nuestro *master layout*, lo siguiente es empezar a construir **vistas** que lo utilicen.

Es decir, que hereden de él todos sus componentes y añadan nuevos elementos, respetando la configuración básica establecida por el *master layout*.

El aspecto de una vista que herede del *master layout* de nuestro ejemplo anterior debería ser como se muestra a continuación...

2. MASTER LAYOUT

```
@extends('master')

@section('title', 'Titulo de la página')

@section('sidebar')
    <p>Esto se añadirá al master sidebar.</p>
@endsection

@section('content')
    <p>Aquí va el contenido de mi página.</p>
@endsection
```

2. MASTER LAYOUT

@extends('master') → Esto indica que la vista hereda del master layout.

@section('title', 'Título de la página') → Aquí se define el contenido de la sección 'title' que en el master layout estaba vacía.

@section('sidebar') → Aquí añade contenido a la sección 'sidebar', que en el master layout no estaba vacía.

@endsection → Se emplea cuando el contenido de una sección ocupa varias líneas de código y no es posible escribirlo en la propia directiva.

3. VARIABLES

Las **variables** a las vistas deben enviarse empaquetadas en un array.

Hay varias formas de lograr esto, todas ellas correctas. Puedes emplear la que te resulte más cómoda o más elegante:

```
// Forma 1: crear el array antes de llamar a la vista
$data['variable1'] = 'valor';
$data['variable2'] = 'valor';
return view('vista', $data);
```

3. VARIABLES

```
// Forma 2: crear un array con el constructor array()
return view('vista', array('variable1'=>'valor', 'variable2'=>'valor'));

// Forma 3: crear un array con corchetes []
return view('vista', ['variable1'=>'valor', 'variable2'=>'valor']);

// Forma 4: usar el método with()
return view('vista')->with(['variable1'=>'valor', 'variable2'=>'valor']);

// Forma 5: usar la función compact()
return view('vista', compact('variable1', 'variable2'));
```

3. VARIABLES

La vista no recibe el array con las variables, sino las **variables ya extraídas** del array. Esto significa que, si le pasas a una vista un par de variables `['variable1'=>'valor', 'variable2'=>'valor']`, en la vista existirán dos variables llamadas `$variable1` y `$variable2`, y no encontrarás ni rastro del array que las contenía. Supongamos que tenemos este método en un controlador:

```
function show() {  
    return view('page', array('name' => 'Rubén Cuevas'));  
}
```

3. VARIABLES

En la vista `/resources/views/page.blade.php` tendrás disponible una variable `$name`, y la puedes usar de este modo:

```
@extends('layouts.master')
@section('title', 'Page Title')
@section('sidebar')
    <p>Esto se añadirá al sidebar del master layout.</p>
@endsection
@section('content')
    <h2>{{ $name }}</h2>
    <p>Este es el contenido de mi página.</p>
@endsection
```

3. VARIABLES

Observa que hemos respetado el *master layout* de ejemplo que hemos creado más arriba. Observa también cómo se hace el `echo` de la variable: nada de abrir y cerrar PHP (`<?php ... ?>`) ni de abrir y cerrar comillas, simplemente se encierra la variable entre llaves dobles (`{{ $variable }}`), y Laravel se encargará de traducir eso al `echo` correspondiente.

La vista queda de ese modo mucho más limpia y, por lo tanto, es más fácil de escribir, leer y depurar.

4. ESTRUCTURAS

Blade admite expresiones **condicionales** para modificar el aspecto de una vista dependiendo del valor de una variable o del estado de la aplicación.

Por supuesto, eso también puede hacerse sin Blade: solo hay que usar un `if` de PHP. Pero con Blade es más fácil y limpio porque, nuevamente, no tendremos que andar abriendo y cerrando PHP ni peleándonos con las comillas del `echo`.

4. ESTRUCTURAS

En el siguiente ejemplo, el contenido de la vista depende de la variable `$day` enviada desde el controlador. Observa como el `if` de PHP se sustituye por la directiva `@if` de Blade.

Controlador:

```
function show() {  
    return view('page', array('name' => 'RdeRumba', 'day' => 'Viernes'));  
}
```

4. ESTRUCTURAS

Vista */resources/views/page.blade.php*:

```
@section('content')  
    <h2>{{ $name }}</h2>  
    <p>Este es el contenido de mi página.</p>  
    @if ($day == 'Viernes')  
        <p>Sesión en Morrissey</p>  
    @else  
        <p>Preparando discos</p>  
    @endif  
@endsection
```

4. ESTRUCTURAS

Igual que existen condicionales, Blade también permite hacer **bucles** para simplificar la generación de vistas.

Por ejemplo, en la siguiente vista recorreremos un array llamado `$bebidas` y mostramos su contenido. Controlador:

```
function show() {  
    $bebidas = array('Cerveza', 'Ron', 'Whisky');  
    return view('page', array('name' => 'Yo', 'day' => 'Viernes', 'bebidas' =>  
$bebidas));  
}
```

4. ESTRUCTURAS

Vista */resources/views/page.blade.php*:

```
@section('content')  
    <h2>{{$name}}</h2>  
    <p>Este es el contenido de mi página.</p>  
    <h2>Estas son las bebidas disponibles:</h2>  
    @foreach ($bebidas as $bebida)  
        {{$bebida}} <br>  
    @endforeach  
@endsection
```

4. ESTRUCTURAS

Desde una vista Blade se puede invocar cualquier función de PHP sin necesidad de abrirlo y cerrarlo (ya sabes: nada de `<?php ... ?>` en las vistas). Por ejemplo, si necesitamos usar la función `date()`:

```
<h2>Ejemplo de llamada a función estándar de PHP</h2>
<p>La fecha del servidor es: {{date('D, M, Y')}}</p>
```

Esto no solo es más limpio, sino también más seguro, porque Blade filtra posibles [ataques por XSS](#) (por cierto: si alguna vez necesitas saltarte la protección contra XSS, puedes usar `{!! ... !!}` en lugar de `{{ ... }}`).

5. DIRECTIVAS HABITUALES

@section → Marca el inicio de una sección de contenido.

@endsection → Marca el final de una sección.

@show → Marca el final de una sección y la muestra.

@yield('section') → Muestra el contenido de una sección, si existe.

@extends('view') → Hereda de una plantilla padre.

@parent → Muestra el contenido de la sección del mismo nombre en la plantilla padre.

5. DIRECTIVAS HABITUALES

@include('view') → Incluye una subvista.

@if - @else - @endif → Condicional.

@for - @endfor → Bucle for clásico.

@foreach - @endforeach → Bucle foreach.

@forelse - @empty - @endforelse → Como foreach, pero con tratamiento de arrays que vienen vacíos (es como un foreach dentro de un if).

5. DIRECTIVAS HABITUALES

@break - **@continue** → Para usar en los bucles.

@switch - **@case** - **@break** → Condicional múltiple.

@isset(\$variable) - **@endisset** → Comprueba si la variable existe.

@auth - **@endauth** → La sección solo se muestra si hay un usuario autenticado en la aplicación.

@php - **@endphp** → Para añadir código PHP plano. Usar con moderación.

5. DIRECTIVAS HABITUALES

{{- Comment -}} → Comentarios (no serán renderizados).

{{ \$variable }} → Equivalente a `echo $variable`, pero no es necesario abrir y cerrar PHP, y nos protegemos contra inyección de JS.

{{ \$variable ?? texto-por-defecto }} → Como el anterior, pero chequea si la variable existe. Si no existe, muestra texto-por-defecto.

\$loop → Es una variable muy útil para usar al depurar un `foreach`. Nos dice si estamos en el primer elemento, en el último, cuántos loops llevamos, etc.

6. FORMULARIOS

Blade también facilita mucho el **tratamiento de los formularios**. Y, por supuesto, filtra por nosotros cualquier código malicioso que traten de colarnos a través de ellos. Vamos a verlo con un ejemplo:

```
<form method="POST" action="{{ route('mi-ruta') }}"> <!--Ruta por su nombre-->
    @csrf    <!-- Para evitar ataques CSRF -->
    <input type="email" name="email"><br>
    <input type="text" name="asunto"><br>
    <textarea name="contenido"></textarea><br>
    <button type="submit">Enviar</button>
</form>
```

6. FORMULARIOS

Recuerda que los métodos PUT, PATCH y DELETE no existen aún en HTML y que Laravel los simula con un campo oculto en el formulario que se crea con la directiva **@method('PUT')** (o PATCH o DELETE). En este formulario no la hemos usado porque se envía por POST.

Lo que sí hemos usado es la directiva **@csrf**. Todos los formularios deberían llevarla. Impide los [ataques por CSRF](#), incrustando un token aleatorio en el formulario que Laravel se encargará de comprobar cuando los datos del formulario regresen al servidor.

En el enrutador definiremos la ruta “mi-ruta” del *action* del formulario y, en el controlador, creamos el método `store()`. Observa cómo recuperamos los datos del formulario a través de la variable `$r`:

```
request("asunto");
```

\$asunto =

6. FORMULARIOS

Otra cosa que Blade facilita muchísimo es la **validación de formularios en el lado del servidor**.

Aquí toca hacer un paréntesis para responder a esta pregunta: ¿dónde deberían validarse los formularios? ¿En el cliente (con HTML y Javascript) o en el servidor (con PHP y Laravel)?

La respuesta es: en los dos sitios. Sobre todo si es un formulario con información sensible.

6. FORMULARIOS

La validación en el cliente es más sencilla de hacer. Comprobar que un campo de texto de un formulario no se envía vacío, por ejemplo, es algo trivial con HTML. Y con Javascript es solo un poco más complicado.

Hacer esas comprobaciones en el servidor resulta más trabajoso: hay que recibir el formulario, comprobar los valores de los campos y, si hay un error, tenemos que volver a enviar el formulario, pero respetando los datos que el usuario ya hubiera tecleado, todo ello junto con un mensaje informando del error.

6. FORMULARIOS

Sí, en el servidor es más complicado. Pero también más seguro. Las validaciones en el cliente pueden desactivarse (puedo usar un navegador antiguo para evitar las comprobaciones por HTML, y puedo deshabilitar temporalmente el motor de Javascript), así que son una primera barrera defensiva muy conveniente pero poco segura.

Las comprobaciones en el servidor son imposibles de saltar para un atacante. Así que deberíamos hacer los dos conjuntos de comprobaciones en todos nuestros formularios. Y aquí es donde Laravel nos ayuda

6. FORMULARIOS

Vamos a retomar la vista del ejemplo anterior y a modificar el formulario para que se valide en el servidor. Tenemos que añadirle algunas cosas:

```
<form method="POST" action="{{ route('mi-ruta') }}">
    @if ($errors->any())
        @foreach ($errors->all() as $error)
            {{ $error }}<br>
        @endforeach
    @endif
    <!-- Resto del formulario igual -->
</form>
```


6. FORMULARIOS

El objeto `$errors` (disponible en todas las vistas) tiene muchos más métodos útiles que puedes consultar en la [documentación](#) de Laravel. Y el controlador quedaría así (observa cómo definimos varias reglas de validación para campos del formulario):

```
public function store() {  
    request->validate([  
        'email' => 'required|email',  
        'asunto' => 'required'  
    ]); // A partir de aquí, se procesa el formulario igual que antes  
}
```

6. FORMULARIOS

Cuando vuelve a cargarse el formulario que contenía un error, suele ser apropiado mostrarlo con los datos que el usuario ya había tecleado. A esto se le llama “repopular” el formulario, y con Blade se hace así (observa el atributo *value* del campo *email*):

```
<form method="POST" action="{{ route('mi-ruta') }}">
    @if ($errors->any()) ...
    @endif
    <input type="email" name="email" value="{{ old('email') }}"><br>
    <!-- Resto del formulario igual -->
</form>
```

7. CSS Y JAVASCRIPT

Laravel ya trae dos archivos (*app.css* y *app.js*) basados en Bootstrap para empezar a trabajar. Para usarlos, basta con añadir esto a la cabecera de nuestras vistas:

```
<link rel="stylesheet" href="/css/app.css">
<script src="js/app.js" defer></script>
```

Si queremos añadir reglas CSS, *NO* debemos editar */public/app.css*, porque es un CSS compilado y minimizado con SASS.

7. CSS Y JAVASCRIPT

Lo correcto para añadir nuestro CSS a ese archivo es:

1. Editar `/resources/sass/app.css`
2. Recompilar este archivo con `SASS` (o con `less` o con `stylus`)

La recompilación se hace con este comando:

```
$ npm run dev
```

O también con:

```
$ yarn dev
```

7. CSS Y JAVASCRIPT

Como no hay tiempo para explicarlo todo, simplemente guarda tu CSS y tu JavaScript en el directorio */public* de Laravel. En ese caso, los archivos no estarían optimizados (tendríamos que optimizarlos a mano, si queremos) y serían accesibles de forma pública, pero es una solución lo suficientemente buena para empezar.

Un consejo: si vas a colocar tu CSS y tu JavaScript en */public*, al menos crea subcarpetas para organizarlo un poco. Por ejemplo, */public/css* y */public/js*.

8. VISTAS DE ERROR PERSONALIZADAS

Por último, te menciono un pequeño truco para darle a tu aplicación un toque más profesional: personalizar las vistas de error.

Simplemente, crea una carpeta `/resources/views/errors`. Todas las vistas que metas ahí dentro se considerarán pantallas de error.

Ahora solo tienes que ponerles los nombres adecuados. Por ejemplo, si creas un archivo llamado `/resources/views/errors/404.blade.php`, esa vista se mostrará cada vez que ocurra un error 404 (página no encontrada). Fácil, ¿no?